# ACN Programming Assignment 2

Socket Programming: TCP and UDP Mini File Transfer

Roll Number 1: CS26MTECH11008
Name 1: SAI CHAITANYA MUPPARAJU

Roll Number 2: CS26MTECH11006
Name 2: GAJJALA JYOTHI SAI HARI TEJA

Department of Computer Science and Engineering
IIT Hyderabad

Date: 31-08-2026

# Part 1 — TCP File Transfer

This part implements a simple FTP-style tool over TCP. A client connects to a server, and can list files on the server, upload a file with put, or download a file with get. Because TCP keeps a connection open and guarantees ordered, lossless delivery, the code does not need to handle missing or out-of-order data itself — the transport layer already takes care of that. To still be sure that a file arrives unchanged, every transfer is checked with a SHA-256 checksum on both ends.

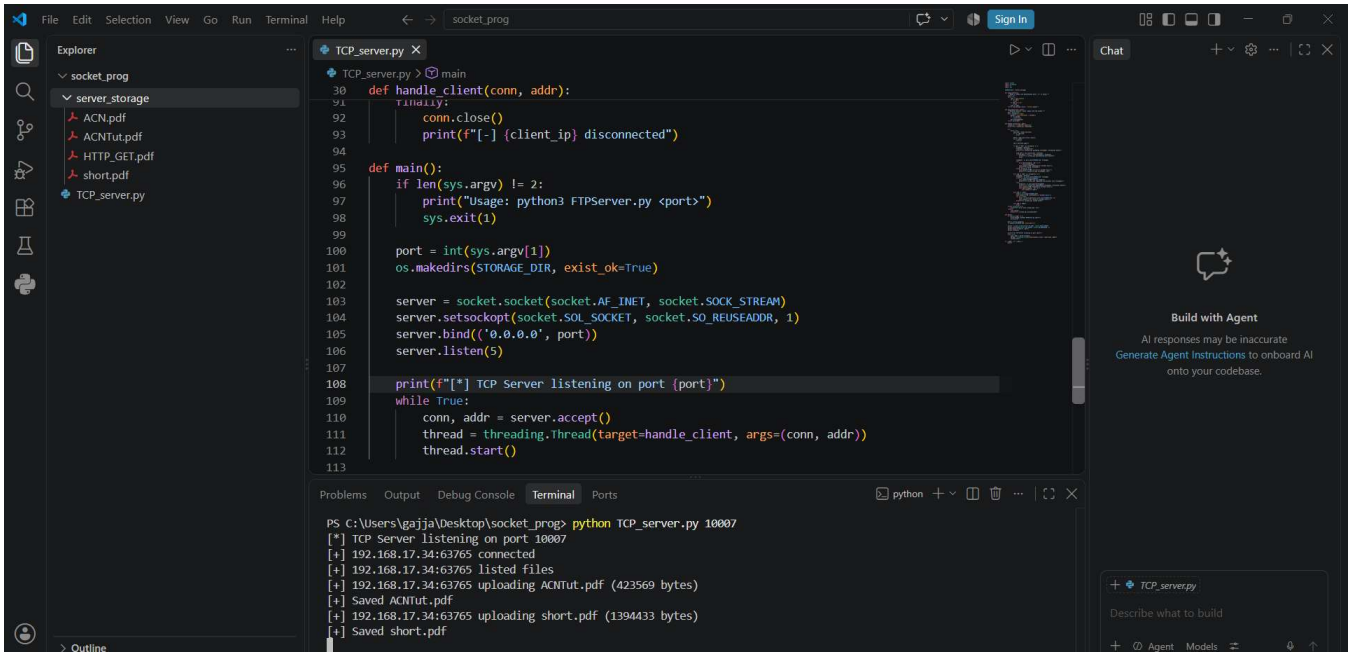
## Uploading and downloading a file

The client first connects to the server and lists what's already stored there. A put command then sends a file across the server writes it to its storage folder and confirms with a save message. A get command does the reverse, the client asks for a named file and the server streams it back.

## TCPFTP Client: PUT

```
PS C:\Users\saich\Desktop\Socket Programming-2> python TCPFTPclient.py 192.168.17.34 10007
✅ Connected to the FTP server at 192.168.17.34:10007
ftp> list
📂 Server contains [3] files:
 - ACN.pdf 1394433
 - ACNTut.pdf 423569
 - HTTP_GET.pdf 119428
ftp> put ACNTut.pdf
🔒 Calculating local file checksum...
   SHA-256: 9c30ce997ef02a565e591c2d67569b34002b186b5e239bf91e94b55463afb48f
⌛ Uploading ACNTut.pdf (423569 bytes)...
✅ Success: Transferred 423569 total bytes cleanly.
ftp> put short.pdf
🔒 Calculating local file checksum...
   SHA-256: e95af85bb0af1392adbf6a3ecb3b87ed6ccd0195a1e830d6b09140bafa8e9ce0
⌛ Uploading short.pdf (1394433 bytes)...
✅ Success: Transferred 1394433 total bytes cleanly.
ftp>
```

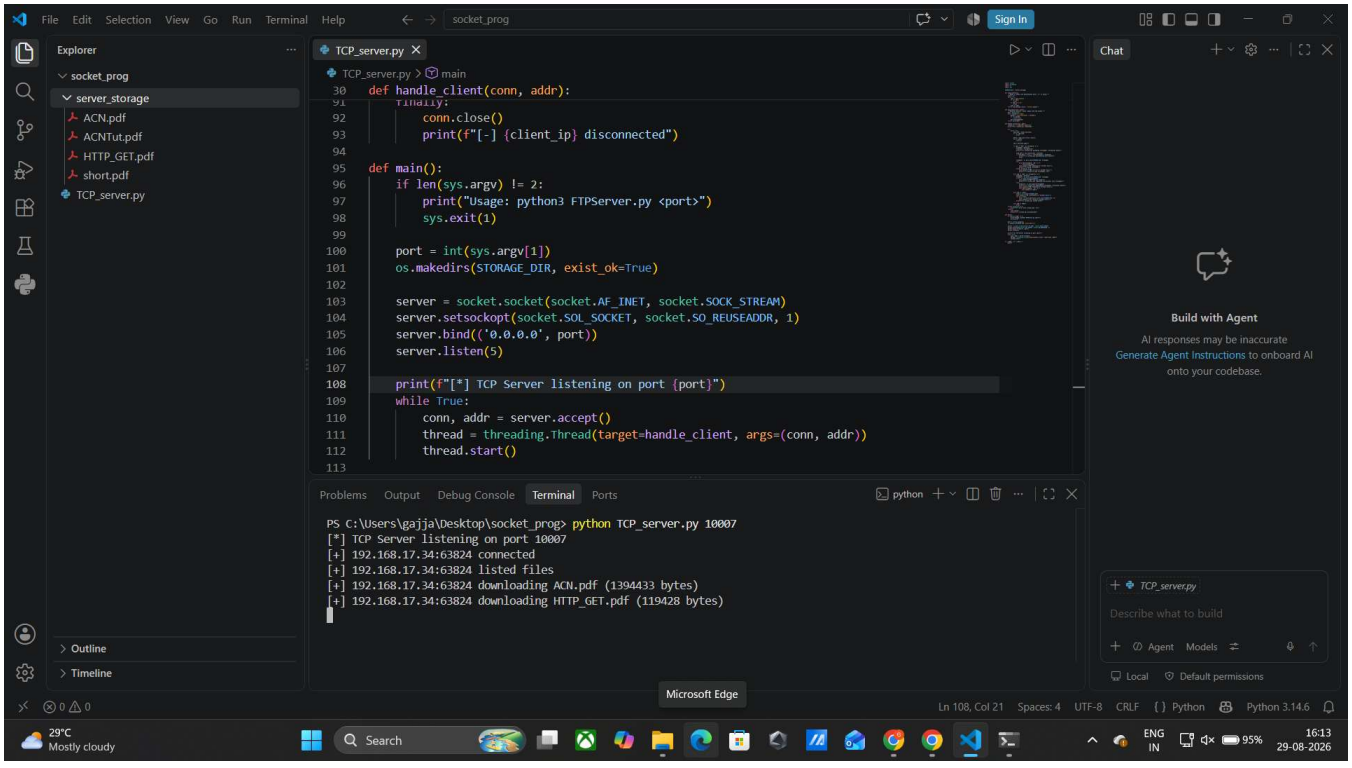
TCPFTP Server:



## TCPFTP CLIENT GET:

```
PS C:\Users\saich\Desktop\Socket Programming-2> python TCPFTPclient.py 192.168.17.34 10007
✅ Connected to the FTP server at 192.168.17.34:10007
ftp> list
📂 Server contains [4] files:
 - ACN.pdf 1394433
 - ACNTut.pdf 423569
 - HTTP_GET.pdf 119428
 - short.pdf 1394433
ftp> get ACN.pdf
⚠️Note: Local file already exists. Auto-renamed destination target to: ACN_1.pdf
✅ Success: Downloaded 1394433 bytes to ACN_1.pdf.
🔒 Verifying integrity checksum of downloaded file...
   SHA-256: e95af85bb0af1392adbf6a3ecb3b87ed6ccd0195a1e830d6b09140bafa8e9ce0
ftp> get HTTP_GET.pdf
⚠️Note: Local file already exists. Auto-renamed destination target to: HTTP_GET_1.pdf
✅ Success: Downloaded 119428 bytes to HTTP_GET_1.pdf.
🔒 Verifying integrity checksum of downloaded file...
   SHA-256: 6d3e72dc5244c229c7aa69a946799be4aa709ea75f4f95710464c72a8e5e7fea
```

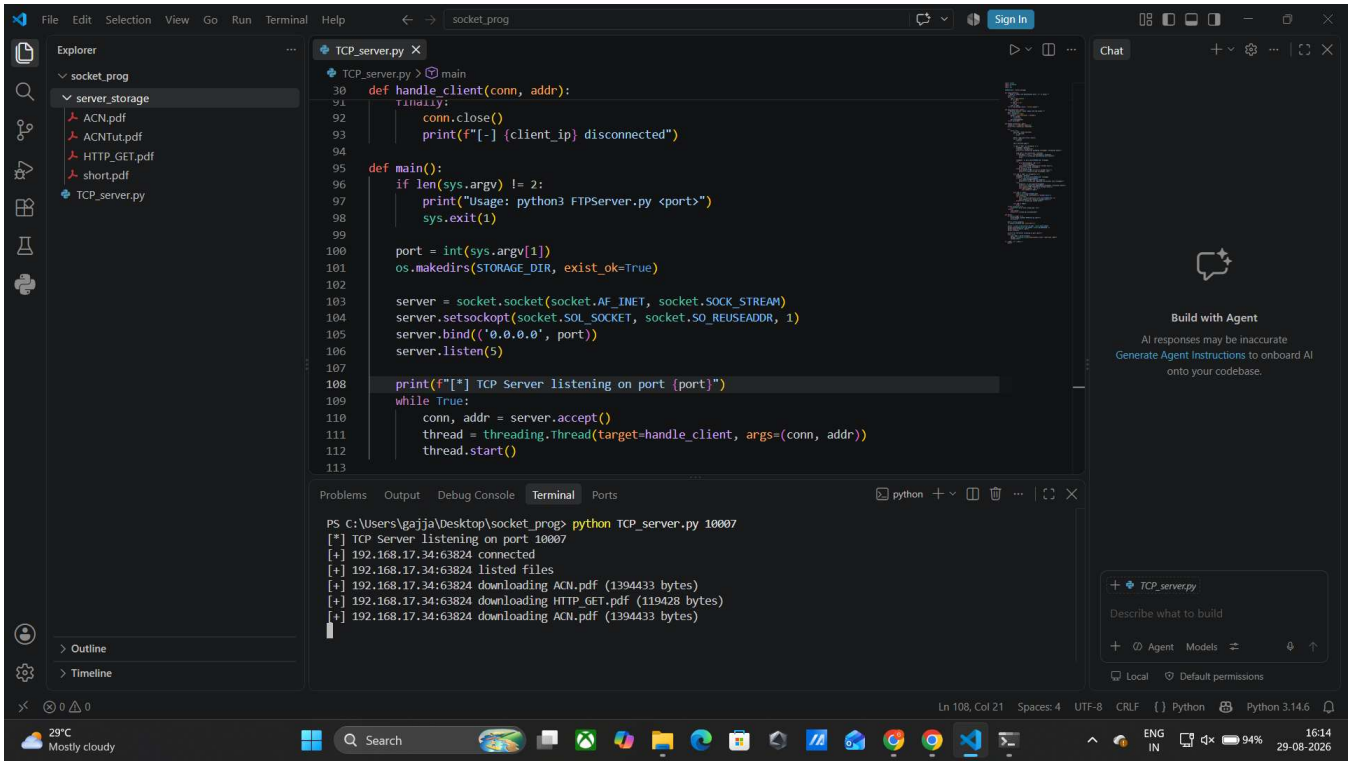
## TCPFTP Server :




## Not overwriting an existing file

If the client asks to download a file it already has locally, the client does not overwrite it. Instead it picks a new name automatically (for example appending a number) and saves the download under that name, so nothing already on disk is lost.

```
ftp> get ACN.pdf
⚠️Note: Local file already exists. Auto-renamed destination target to: ACN_2.pdf
✅ Success: Downloaded 1394433 bytes to ACN_2.pdf.
🔒 Verifying integrity checksum of downloaded file...
   SHA-256: e95af85bb0af1392adbf6a3ecb3b87ed6ccd0195a1e830d6b09140bafa8e9ce0
ftp>
```




## Checking that the file arrived correctly

Before uploading, the client computes a SHA-256 hash of the local file. After a download finishes, it hashes the received copy as well and compares the two values. A match confirms the bytes were not corrupted anywhere along the way.

```
PS C:\Users\saich\Desktop\Socket Programming-2> python TCPFTPclient.py 192.168.17.34 10007
✅ Connected to the FTP server at 192.168.17.34:10007
ftp> put short.pdf
🔒 Calculating local file checksum...
   SHA-256: e95af85bb0af1392adbf6a3ecb3b87ed6ccd0195a1e830d6b09140bafa8e9ce0
⏳ Uploading short.pdf (1394433 bytes)...
✅ Success: Transferred 1394433 total bytes cleanly.
ftp> get short.pdf
⚠️Note: Local file already exists. Auto-renamed destination target to: short_1.pdf
✅ Success: Downloaded 1394433 bytes to short_1.pdf.
🔒 Verifying integrity checksum of downloaded file...
   SHA-256: e95af85bb0af1392adbf6a3ecb3b87ed6ccd0195a1e830d6b09140bafa8e9ce0
ftp>
```

## Serving more than one client at a time

The server spins up a new thread for every client that connects, so several clients can upload and download at the same time without blocking one another. The two runs below show two separate clients talking to the same server concurrently — one doing a put followed by a get, and the other doing the same with different files.

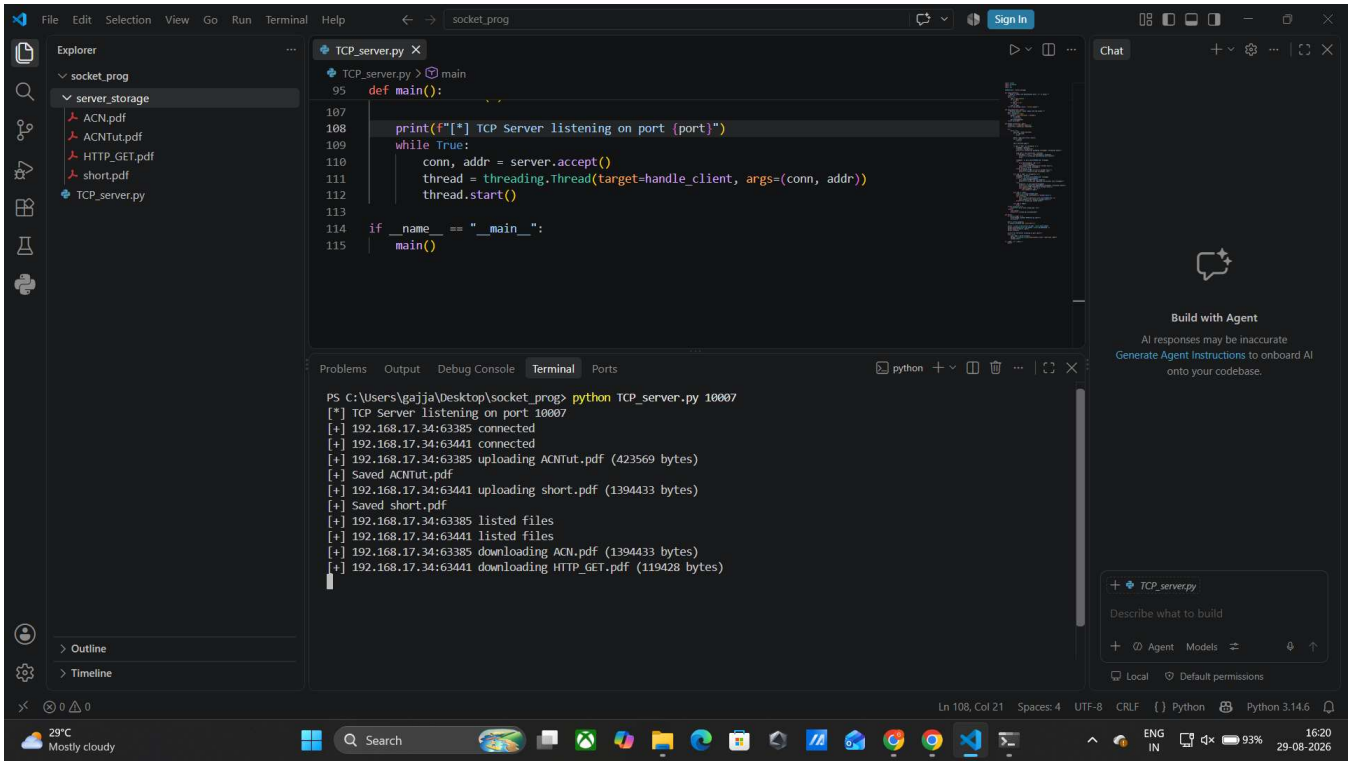
**TCPFTP Client1:**

```
PS C:\Users\saich\Desktop\Socket Programming-2> python TCPFTPclient.py 192.168.17.34 10007
 ✅ Connected to the FTP server at 192.168.17.34:10007
ftp> put ACNTut.pdf
 🔒 Calculating local file checksum...
    SHA-256: 9c30ce997ef02a565e591c2d67569b34002b186b5e239bf91e94b55463afb48f
 ⏳ Uploading ACNTut.pdf (423569 bytes)...
 ✅ Success: Transferred 423569 total bytes cleanly.
ftp> list
 📂 Server contains [4] files:
  - ACN.pdf 1394433
  - ACNTut.pdf 423569
  - HTTP_GET.pdf 119428
  - short.pdf 1394433
ftp> get ACN.pdf
 ⚠️Note: Local file already exists. Auto-renamed destination target to: ACN_3.pdf
 ✅ Success: Downloaded 1394433 bytes to ACN_3.pdf.
 🔒 Verifying integrity checksum of downloaded file...
    SHA-256: e95af85bb0af1392adbf6a3ecb3b87ed6ccd0195a1e830d6b09140bafa8e9ce0
ftp>
```

**TCPFTP Client2:**

```
PS C:\Users\saich\Desktop\Socket Programming-2> python TCPFTPclient.py 192.168.17.34 10007
 ✅ Connected to the FTP server at 192.168.17.34:10007
ftp> put short.pdf
 🔒 Calculating local file checksum...
    SHA-256: e95af85bb0af1392adbf6a3ecb3b87ed6ccd0195a1e830d6b09140bafa8e9ce0
 ⏳ Uploading short.pdf (1394433 bytes)...
 ✅ Success: Transferred 1394433 total bytes cleanly.
ftp> list
 📂 Server contains [4] files:
  - ACN.pdf 1394433
  - ACNTut.pdf 423569
  - HTTP_GET.pdf 119428
  - short.pdf 1394433
ftp> get HTTP_GET.pdf
 ⚠️Note: Local file already exists. Auto-renamed destination target to: HTTP_GET_2.pdf
 ✅ Success: Downloaded 119428 bytes to HTTP_GET_2.pdf.
 🔒 Verifying integrity checksum of downloaded file...
    SHA-256: 6d3e72dc5244c229c7aa69a946799be4aa709ea75f4f95710464c72a8e5e7fea
ftp>
```

# TCPFTP Server :



# Part 2 — UDP File Transfer with Reliability

UDP does not guarantee delivery, ordering, or even that a packet arrives at all, so this part builds a small reliability layer on top of it: the file is split into fixed-size chunks, each chunk is numbered, and the receiver acknowledges every chunk it gets. If the sender does not see an acknowledgement in time, it resends that chunk, up to a retry limit. The same put / get / list commands as Part 1 are used, and every completed transfer is verified with an MD5 hash.

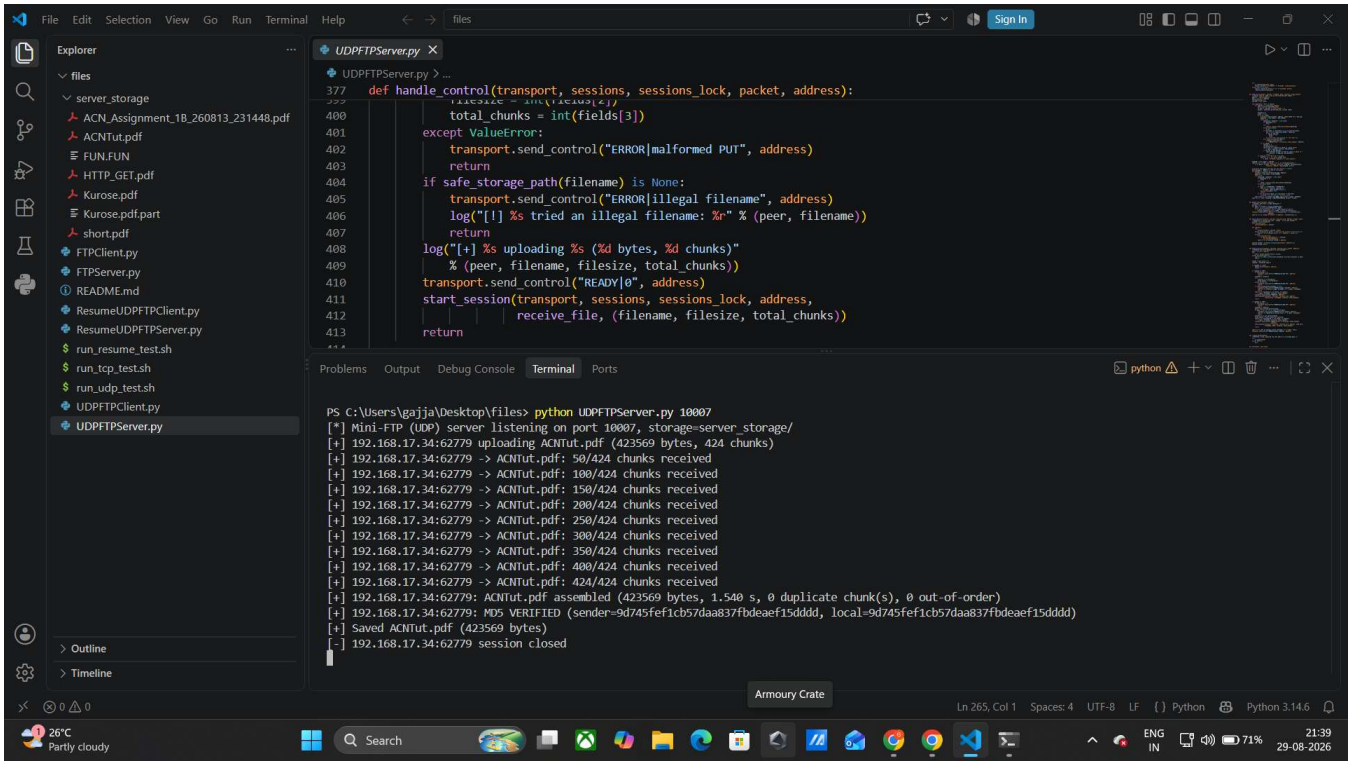
## Baseline run with no packet loss

With 0% simulated loss, every chunk is acknowledged on the first try, so there are zero retransmissions and the transfer runs at close to the network's real speed.

### UDPFTP Client:

```
PS C:\Users\saich\Desktop\Socket Programming-2> python UDPFTPclient.py 192.168.17.34  10007
Commands: put <file> | get <file> | list | quit
udp-ftp> put ACNTut.pdf
[*] PUT ACNTut.pdf (423569 bytes, 424 chunks), local MD5 = 9d745fef1cb57daa837fbdeaef15dddd
[+] 50/424 chunks acknowledged
[+] 100/424 chunks acknowledged
[+] 150/424 chunks acknowledged
[+] 200/424 chunks acknowledged
[+] 250/424 chunks acknowledged
[+] 300/424 chunks acknowledged
[+] 350/424 chunks acknowledged
[+] 400/424 chunks acknowledged
[+] 424/424 chunks acknowledged
[=] PUT ACNTut.pdf | 423569 bytes | 1.541 s | 0 retransmitted chunk(s) | 268.51 KB/s | VERIFIED
udp-ftp>
```

### UDPFTP Server :



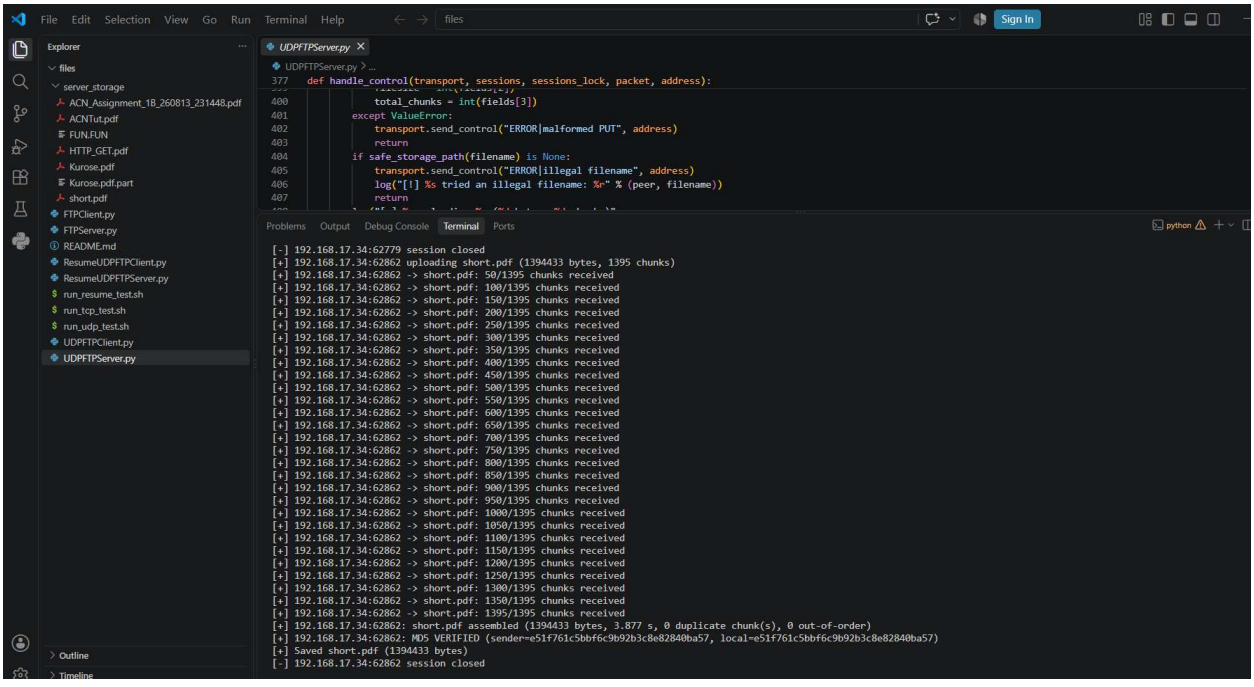
## A larger file (over 1 MB)

The same put command was repeated with a bigger file to check that chunking and reassembly still work correctly once **there are well over a thousand chunks to track.**

### UDPFTP Client:

```
PS C:\Users\saich\Desktop\Socket Programming-2> python UDPFTPclient.py 192.168.17.34  10007
Commands: put <file> | get <file> | list | quit
udp-ftp> put short.pdf
[*] PUT short.pdf (1394433 bytes, 1395 chunks), local MD5 = e51f761c5bbf6c9b92b3c8e82840ba57
[+] 50/1395 chunks acknowledged
[+] 100/1395 chunks acknowledged
[+] 150/1395 chunks acknowledged
[+] 200/1395 chunks acknowledged
[+] 250/1395 chunks acknowledged
[+] 300/1395 chunks acknowledged
[+] 350/1395 chunks acknowledged
[+] 400/1395 chunks acknowledged
[+] 450/1395 chunks acknowledged
[+] 500/1395 chunks acknowledged
[+] 550/1395 chunks acknowledged
[+] 600/1395 chunks acknowledged
[+] 650/1395 chunks acknowledged
[+] 700/1395 chunks acknowledged
[+] 750/1395 chunks acknowledged
[+] 800/1395 chunks acknowledged
[+] 850/1395 chunks acknowledged
[+] 900/1395 chunks acknowledged
[+] 950/1395 chunks acknowledged
[+] 1000/1395 chunks acknowledged
[+] 1050/1395 chunks acknowledged
[+] 1100/1395 chunks acknowledged
[+] 1150/1395 chunks acknowledged
[+] 1200/1395 chunks acknowledged
[+] 1250/1395 chunks acknowledged
[+] 1300/1395 chunks acknowledged
[+] 1350/1395 chunks acknowledged
[+] 1395/1395 chunks acknowledged
[=] PUT short.pdf | 1394433 bytes | 3.881 s | 0 retransmitted chunk(s) | 350.86 KB/s | VERIFIED
udp-ftp>
```

### UDPFTP Server :



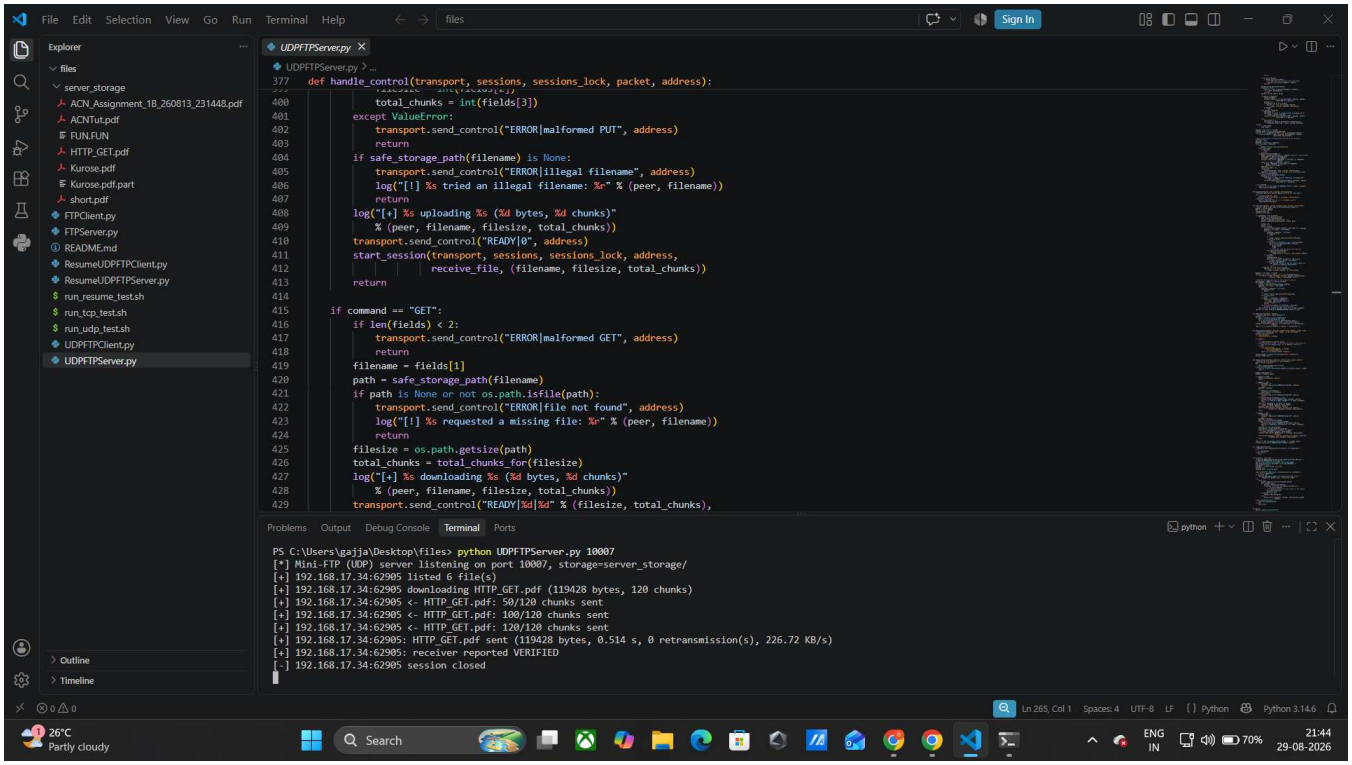
## Downloading files

The get command works the same way in reverse: the server chunks the file it is sending, and the client acknowledges each chunk as it arrives, both for a small file and for one over 1 MB.

### UDPFTP Client:

```
PS C:\Users\saich\Desktop\Socket Programming-2> python UDPFTPclient.py 192.168.17.34  10007
Commands: put <file> | get <file> | list | quit
udp-ftp> list
[*] 6 file(s) on the server:
    ACNTut.pdf                              423569 bytes
    ACN_Assignment_1B_260813_231448.pdf     1394433 bytes
    FUN.FUN                                 45 bytes
    HTTP_GET.pdf                            119428 bytes
    Kurose.pdf                              19661364 bytes
    short.pdf                               1394433 bytes
udp-ftp> get HTTP_GET.pdf
[*] HTTP_GET.pdf already exists locally; saving as HTTP_GET (1).pdf
[*] GET HTTP_GET.pdf (119428 bytes, 120 chunks) -> HTTP_GET (1).pdf
[+] 50/120 chunks received
[+] 100/120 chunks received
[+] 120/120 chunks received
[*] 0 duplicate chunk(s), 0 out-of-order chunk(s)
[=] GET HTTP_GET.pdf | 119428 bytes | 0.501 s | 0 retransmitted chunk(s) | 232.83 KB/s | VERIFIED
[+] Local MD5 = f89433fcbe726efe91d2e41c734465cf
udp-ftp>
```

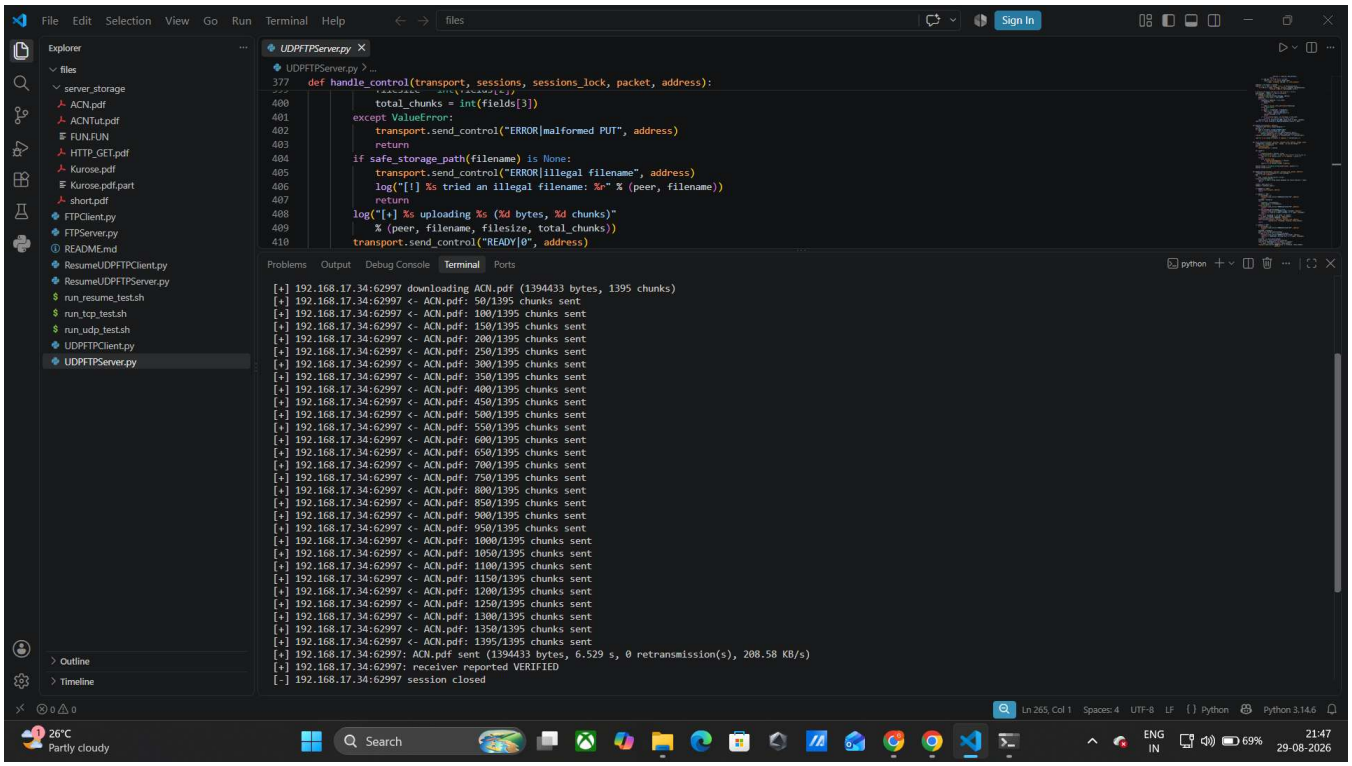
### UDPFTP Server :



## UDPFTP client :

```
PS C:\Users\saich\Desktop\Socket Programming-2> python UDPFTPclient.py 192.168.17.34  10007
Commands: put <file> | get <file> | list | quit
udp-ftp> get ACN.pdf
[*] ACN.pdf already exists locally; saving as ACN (1).pdf
[*] GET ACN.pdf (1394433 bytes, 1395 chunks) -> ACN (1).pdf
[+] 50/1395 chunks received
[+] 100/1395 chunks received
[+] 150/1395 chunks received
[+] 200/1395 chunks received
[+] 250/1395 chunks received
[+] 300/1395 chunks received
[+] 350/1395 chunks received
[+] 400/1395 chunks received
[+] 450/1395 chunks received
[+] 500/1395 chunks received
[+] 550/1395 chunks received
[+] 600/1395 chunks received
[+] 650/1395 chunks received
[+] 700/1395 chunks received
[+] 750/1395 chunks received
[+] 800/1395 chunks received
[+] 850/1395 chunks received
[+] 900/1395 chunks received
[+] 950/1395 chunks received
[+] 1000/1395 chunks received
[+] 1050/1395 chunks received
[+] 1100/1395 chunks received
[+] 1150/1395 chunks received
[+] 1200/1395 chunks received
[+] 1250/1395 chunks received
[+] 1300/1395 chunks received
[+] 1350/1395 chunks received
[+] 1395/1395 chunks received
[*] 0 duplicate chunk(s), 0 out-of-order chunk(s)
[=] GET ACN.pdf | 1394433 bytes | 6.530 s | 0 retransmitted chunk(s) | 208.54 KB/s | VERIFIED
[+] Local MD5 = e51f761c5bbf6c9b92b3c8e82840ba57
udp-ftp>
```

## UDPFTP Server:



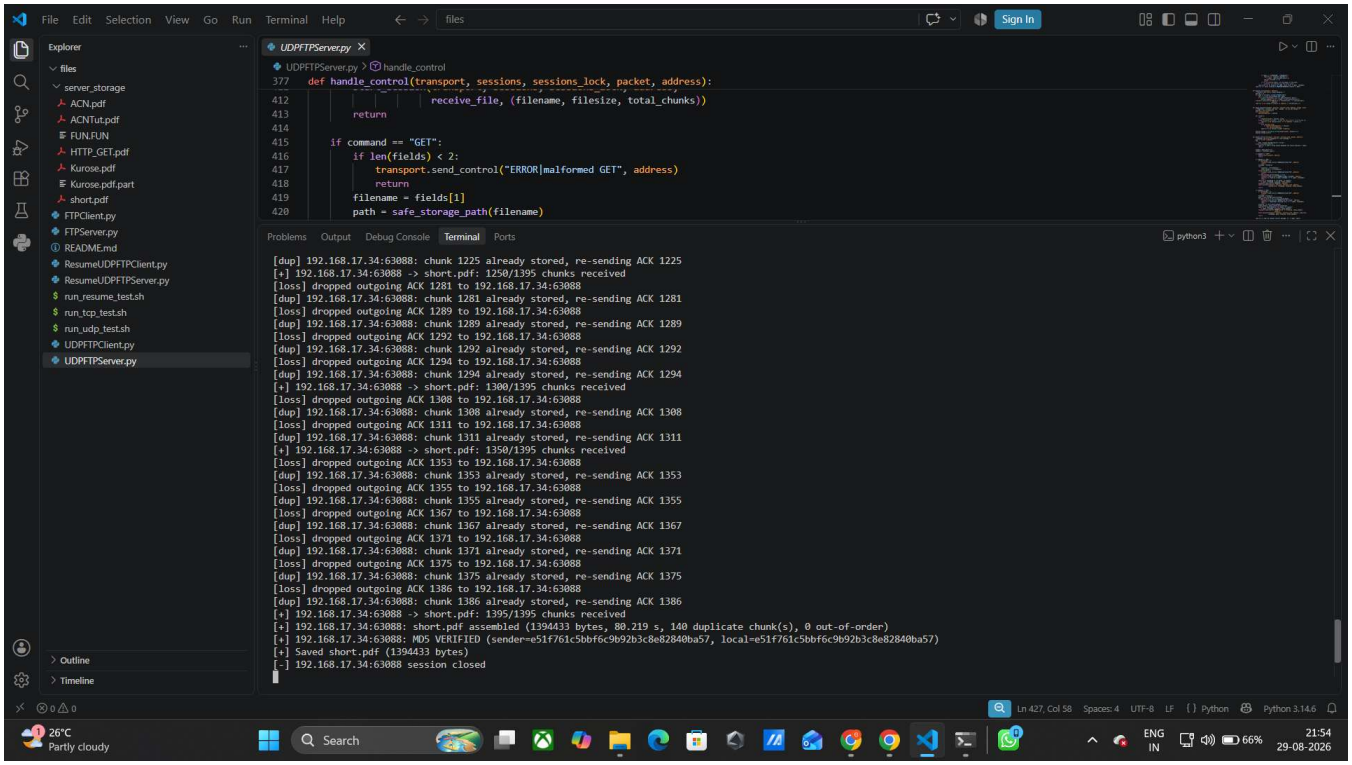

Introducing packet loss

To test the retry logic, packet loss was simulated at two levels: 10% and 20%. At these rates, some chunks (or their acknowledgements) are deliberately dropped, forcing the sender to time out and resend. Each affected chunk is retried up to 10 times before giving up.

## 10% loss — upload (PUT) (UDPFTP Client)

```
PS C:\Users\saich\Desktop\Socket Programming-2> python UDPFTPclient.py 192.168.17.34  10007
[+] 1150/1395 chunks acknowledged
[rtx] timeout on chunk 1153, retry 1/10
[rtx] timeout on chunk 1161, retry 1/10
[rtx] timeout on chunk 1167, retry 1/10
[rtx] timeout on chunk 1171, retry 1/10
[rtx] timeout on chunk 1173, retry 1/10
[rtx] timeout on chunk 1173, retry 2/10
[rtx] timeout on chunk 1175, retry 1/10
[rtx] timeout on chunk 1194, retry 1/10
[+] 1200/1395 chunks acknowledged
[rtx] timeout on chunk 1225, retry 1/10
[+] 1250/1395 chunks acknowledged
[rtx] timeout on chunk 1281, retry 1/10
[rtx] timeout on chunk 1289, retry 1/10
[rtx] timeout on chunk 1292, retry 1/10
[rtx] timeout on chunk 1294, retry 1/10
[+] 1300/1395 chunks acknowledged
[rtx] timeout on chunk 1308, retry 1/10
[rtx] timeout on chunk 1311, retry 1/10
[+] 1350/1395 chunks acknowledged
[rtx] timeout on chunk 1353, retry 1/10
[rtx] timeout on chunk 1355, retry 1/10
[rtx] timeout on chunk 1367, retry 1/10
[rtx] timeout on chunk 1371, retry 1/10
[rtx] timeout on chunk 1375, retry 1/10
[rtx] timeout on chunk 1386, retry 1/10
[+] 1395/1395 chunks acknowledged
[=] PUT short.pdf | 1394433 bytes | 80.224 s | 140 retransmitted chunk(s) | 16.97 KB/s | VERIFIED
udp-ftp>
```

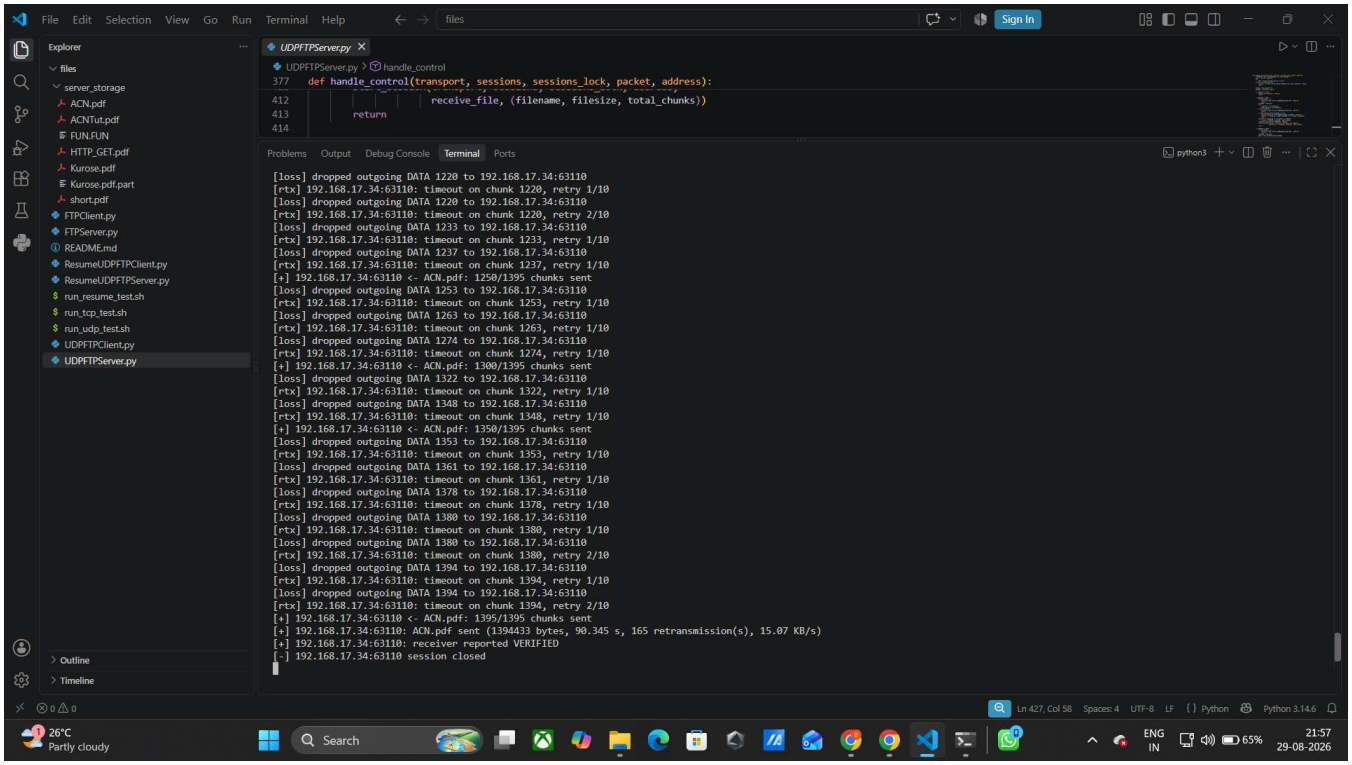
## UDPFTP Server:



## 10% loss — download (GET) (UDPFTP Client)

```
PS C:\Users\saich\Desktop\Socket Programming-2> python UDPFTPclient.py 192.168.17.34  10007
[+] 200/1395 chunks received
[+] 250/1395 chunks received
[+] 300/1395 chunks received
[+] 350/1395 chunks received
[+] 400/1395 chunks received
[+] 450/1395 chunks received
[+] 500/1395 chunks received
[+] 550/1395 chunks received
[+] 600/1395 chunks received
[+] 650/1395 chunks received
[+] 700/1395 chunks received
[+] 750/1395 chunks received
[+] 800/1395 chunks received
[+] 850/1395 chunks received
[+] 900/1395 chunks received
[+] 950/1395 chunks received
[+] 1000/1395 chunks received
[+] 1050/1395 chunks received
[+] 1100/1395 chunks received
[+] 1150/1395 chunks received
[+] 1200/1395 chunks received
[+] 1250/1395 chunks received
[+] 1300/1395 chunks received
[+] 1350/1395 chunks received
[+] 1395/1395 chunks received
[*] 0 duplicate chunk(s), 0 out-of-order chunk(s)
[=] GET ACN.pdf | 1394433 bytes | 90.350 s | 0 retransmitted chunk(s) | 15.07 KB/s | VERIFIED
[+] Local MD5 = e51f761c5bbf6c9b92b3c8e82840ba57
udp-ftp>
```

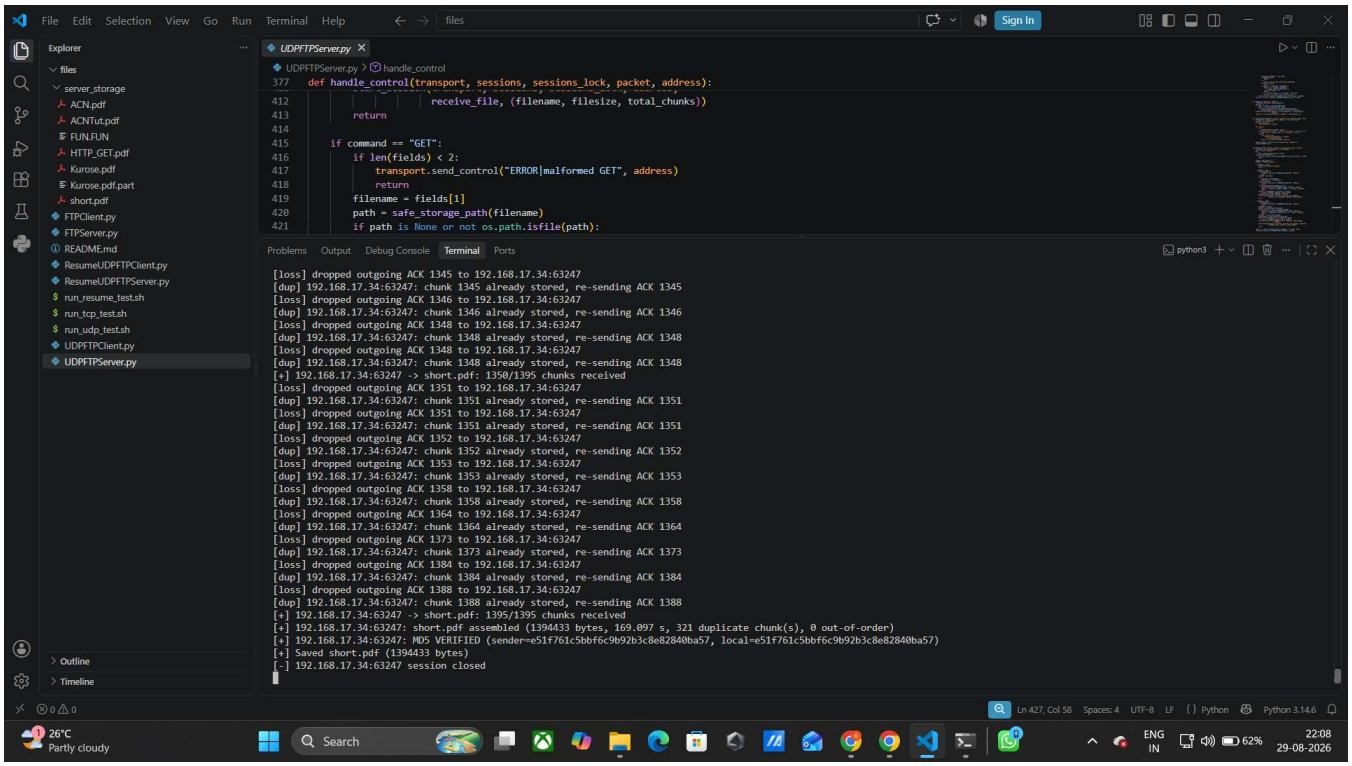
**UDPFTP Server:**



## 20% loss — upload (PUT) (UDPFTP Client)

```
[rtx] timeout on chunk 1348, retry 2/10
[+] 1350/1395 chunks acknowledged
[rtx] timeout on chunk 1351, retry 1/10
[rtx] timeout on chunk 1351, retry 2/10
[rtx] timeout on chunk 1352, retry 1/10
[rtx] timeout on chunk 1353, retry 1/10
[rtx] timeout on chunk 1358, retry 1/10
[rtx] timeout on chunk 1364, retry 1/10
[rtx] timeout on chunk 1373, retry 1/10
[rtx] timeout on chunk 1384, retry 1/10
[rtx] timeout on chunk 1388, retry 1/10
[+] 1395/1395 chunks acknowledged
[=] PUT short.pdf | 1394433 bytes | 169.101 s | 321 retransmitted chunk(s) | 8.05 KB/s | VERIFIED
udp-ftp>
```

**UDPFTP Server:**

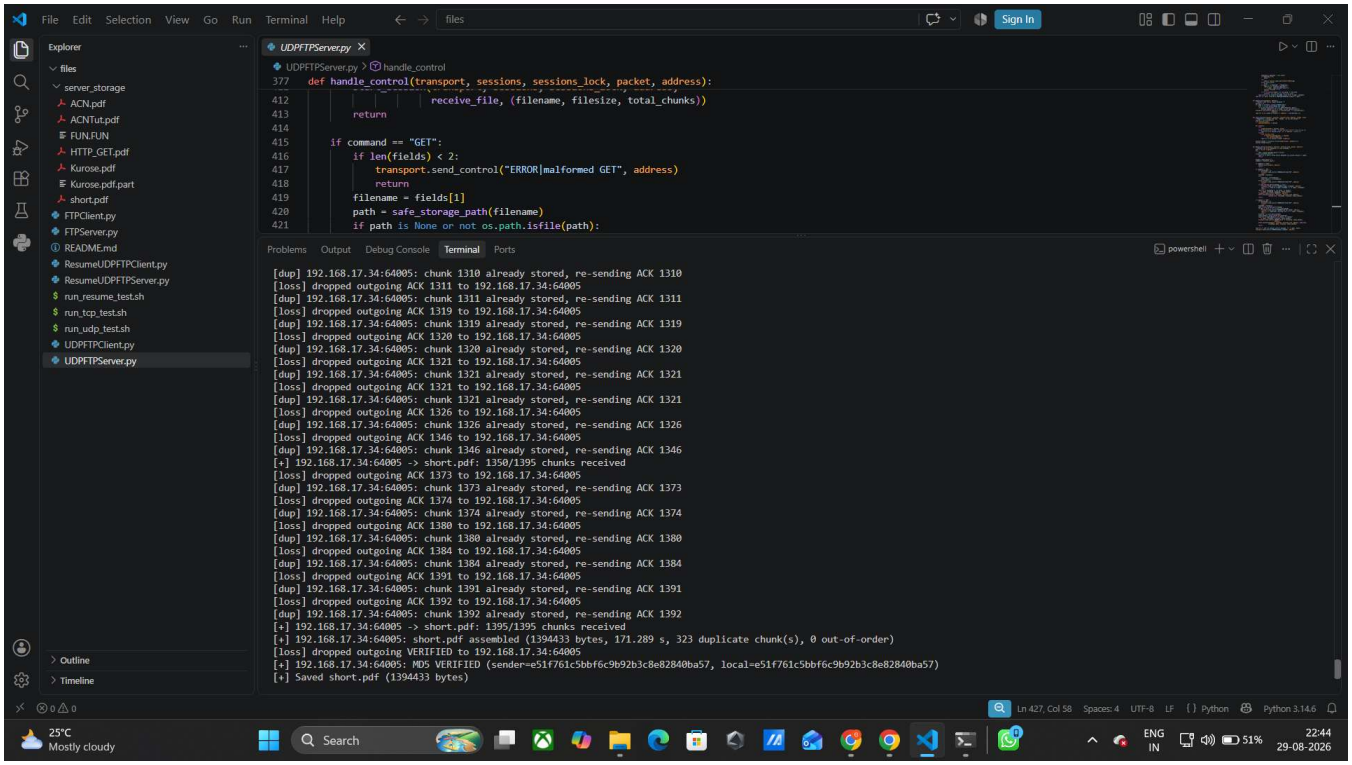



## 20% loss — download (GET)

**UDPFTP Client:**

```
[rtx] timeout on chunk 1346, retry 1/10
[+] 1350/1395 chunks acknowledged
[rtx] timeout on chunk 1373, retry 1/10
[rtx] timeout on chunk 1374, retry 1/10
[rtx] timeout on chunk 1380, retry 1/10
[rtx] timeout on chunk 1384, retry 1/10
[rtx] timeout on chunk 1391, retry 1/10
[rtx] timeout on chunk 1392, retry 1/10
[+] 1395/1395 chunks acknowledged
[rtx] no verdict for DONE, retry 1/5
[rtx] no verdict for DONE, retry 2/5
[rtx] no verdict for DONE, retry 3/5
[rtx] no verdict for DONE, retry 4/5
[rtx] no verdict for DONE, retry 5/5
[=] PUT short.pdf | 1394433 bytes | 171.295 s | 323 retransmitted chunk(s) | 7.95 KB/s | NO VERDICT
udp-ftp>
```

**UDPFTP Server:**



# Loss vs. Performance Summary

The numbers reported by the client at the end of each run are collected below, for both directions of transfer and all three loss levels tested.

| Request | File (bytes) | Loss | Time (s) | Retransmitted chunks | Throughput (KB/s) |
|---|---|---|---|---|---|
| PUT | short.pdf (1,394,433) | 0% | 3.88 | 0 | 350.86 |
| PUT | short.pdf (1,394,433) | 10% | 80.22 | 140 | 16.97 |
| PUT | short.pdf (1,394,433) | 20% | 169.10 | 321 | 8.05 |
| GET | ACN.pdf (1,394,433) | 0% | 6.53 | 0 | 208.54 |
| GET | ACN.pdf (1,394,433) | 10% | 90.35 | 165 | 15.07 |
| GET | ACN.pdf (1,394,433) | 20% | 171.30 | 323 | 7.95 |

## What the numbers show

Loss and performance move in lockstep here. Going from 0% to 10% loss already pushes retransmissions from zero up into the hundreds, and transfer time jumps by more than 20 times. Doubling the loss again to 20% roughly doubles both the retransmission count and the time taken, while throughput keeps falling — from the hundreds of KB/s at 0% loss down to single-digit KB/s at 20%. In short: every dropped packet costs a timeout-and-resend cycle, and those cycles add up fast once loss gets past a few percent.

# Work Distribution

The work below was split between the two of us as follows.

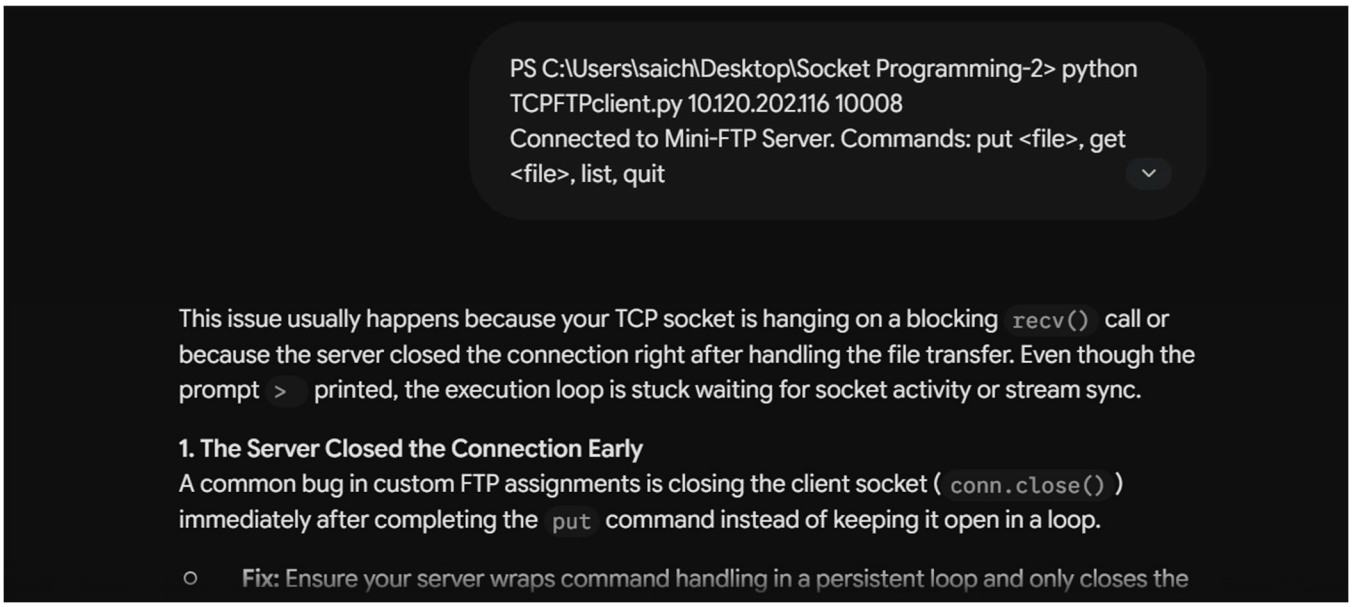
## Partner 1: GAJJALA JYOTHI SAI HARI TEJA — CS26MTECH11006

| Task | Contribution | Challenges faced |
|---|---|---|
| Design & setup | Set up the TCP server skeleton and the UDP server skeleton, including socket binding and the request-handling loop for both. | Getting the framing right so messages didn't get mixed up on the socket stream. |
| TCP server-side integrity | Implemented the server-side SHA-256 checksum handling and the storage logic for incoming files. | Making sure the hash was computed after the file was fully flushed to disk. |
| Multi-client TCP server | Added threading on the TCP server so multiple clients could connect at once. | A race condition the first time two clients wrote to the log at the same moment. |
| UDP server reliability | Implemented the server-side chunking, acknowledgement sending, and reassembly logic for the UDP server. | Handling duplicate and out-of-order chunk arrivals correctly during reassembly. |
| Testing (server side) | Ran and screenshotted the server-side logs for the small-file, large-file, and multi-client scenarios on both TCP and UDP. | None major. |

## Partner 2: SAI CHAITANYA MUPPARAJU— CS26MTECH11008

| Task | Contribution | Challenges faced |
|---|---|---|
| Design & setup | Set up the TCP client and UDP client skeletons, and chose a chunk size that balanced overhead against packet-loss risk. | Choosing a chunk size that balanced overhead against packet-loss risk. |
| TCP client integrity checks | Added client-side SHA-256 checksum verification and the auto-rename logic for existing files. | Making sure the hash was computed after the file was fully flushed to disk. |
| UDP client reliability layer | Implemented the client-side stop-and-wait acknowledgement and retry logic with a retry cap. | An early bug caused an infinite retry loop when a duplicate ACK arrived. |
| Loss simulation | Built the artificial packet/ACK drop feature used for the 10% and 20% loss tests. | Keeping the drop simulation deterministic enough to reproduce results. |
| Testing (client side) | Ran all six loss-rate experiments from the client side and put together the comparison table. | Long run times at 20% loss made repeated testing slow. |

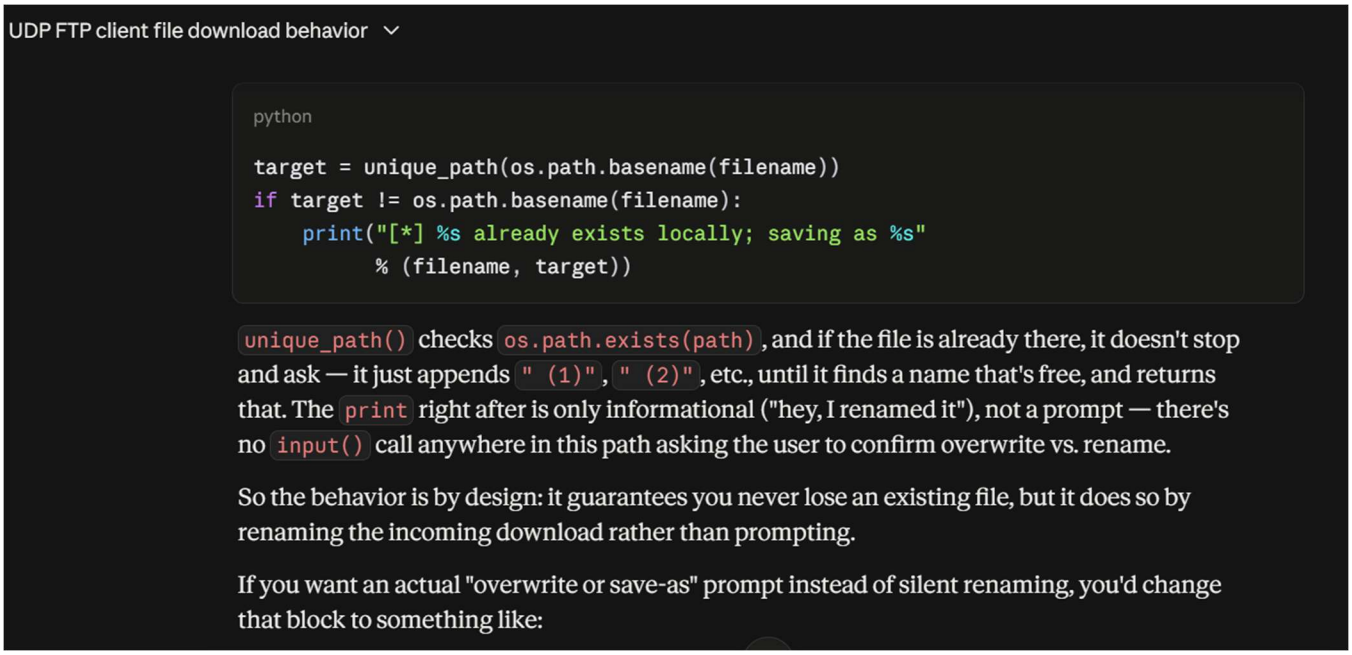
# LLM USAGE:




https://gemini.google.com/app/6684c1dba3fb918b?hl=en-IN

https://claude.ai/chat/f61dcb1c-11b1-4595-9b29-19f760d7c652

https://chatgpt.com/share/6a95aa88-0148-83e8-80c0-7bedb6c0ed78



**ANTI-PLAGIARISM STATEMENT:**

We certify that this assignment/report is the result of our collaborative work, based on our collective study and research. All sources, including books, articles, software, datasets, reports, and communications, have been properly acknowledged. This work has not been previously submitted for assessment in any other course unless specific permission was granted by all involved instructors. We also acknowledge the use of AI tools, such as LLMs (e.g., ChatGPT,Gemini), for assistance in refining this assignment, if used. We have ensured that their usage complies with the academic integrity policies of this course. We pledge to uphold the principles of honesty, integrity, and responsibility at CSE@IITH. Additionally, we understand our duty to report any violations of academic integrity by others if we become aware of them.

Names <Roll Nos>:

SAI CHAITANYA MUPPARAJU<CS26MTECH11008>

GAJJALA JYOTHI SAI HARI TEJA<CS26MTECH11006>

Date : 31-08-2026

Signatures : Sai Chaitanya

Hari Teja